Studying Karpathy's Autoresearch

A Systematic Research Plan

Effect of `program.md` on Autonomous ML Experimentation

Christian Block

U Goethe
Research Course — Supervisor: Ramesh

May 25, 2026

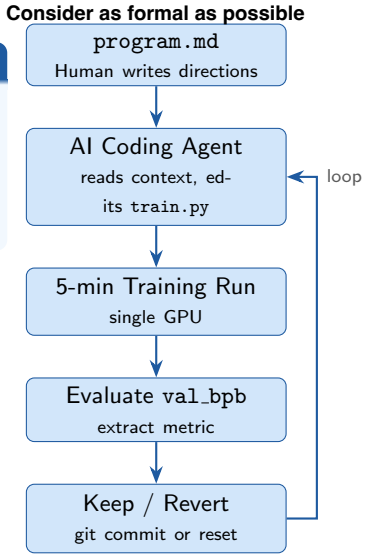
Outline

What is Karpathy's Autoresearch?

Core Idea

An AI coding agent (e.g. Claude Code) is given a minimal ML training setup and a Markdown instruction file. It runs experiments *autonomously*, keeps improvements, and discards failures — overnight, without human intervention.

- Released March 7, 2026 by Andrej Karpathy
- $>$ 66,000 GitHub stars within one month
- ˜100 experiments per overnight run on a single GPU
- Each experiment: **5-minute** fixed time budget
- Loop coined *"The Karpathy Loop"* by Fortune magazine



The Three-File Architecture

prepare.py

Fixed. Agent cannot touch it.

Data preparation, tokenizer, dataloader, evaluation logic.

Guarantees a stable baseline.

train.py

Mutable. The agent edits this.

Model architecture, optimizer, LR schedule, training loop.

The hypothesis space lives here.

program.md

Human-controlled. The research lever.

Natural-language instructions: scope, constraints, strategy, evaluation criteria.

The object of our study.

Research question: How does the content and structure of `program.md` shape the trajectory, efficiency, and quality of autonomous experimentation?

Why This is an Interesting Research Question

Scientific framing

- `program.md` is the only human-controlled variable
- All other conditions are held fixed
- Natural quasi-experimental setup

Open critique in the community

- Works well for narrow, scorable tasks
- Harder for open-ended or creative tasks
- Hyperparameter tuning is the canonical hard case

Practical relevance

- Researchers must write better `program.md`
- No systematic study exists yet
- Results generalize to other agentic engineering settings

Research Objectives

O1 Random Forest (in depth)
Understand how different `program.md` formulations steer autoresearch.

O2 Gradient Boosting (in depth)
Replicate the analysis for gradient boosted trees.

O3 Hyperparameter tuning comparison
Compare autoresearch-guided HP tuning for RF vs GBM.

O4 Open-ended tasks (extension)
Evaluate autoresearch on tasks without a scalar evaluation metric.

Research Questions

RQ1 — Structural effects

How do structural choices in `program.md` affect the number and magnitude of improvements?

RQ2 — Algorithm-specific behaviour

Do the same `program.md` patterns produce different agent strategies for RF vs GBM?

RQ3 — Hyperparameter tuning efficiency

Which `program.md` formulation leads to the most sample-efficient search?

RQ4 — Open-ended tasks

Under what conditions does autoresearch fail or succeed without a single scalar metric?

Study 1: Random Forest — Setup

Problem instance

- Dataset: fixed benchmark dataset
- Model: `RandomForestClassifier`
- Metric: validation AUC / accuracy
- Time budget: 5 minutes per experiment

What varies

Only `program.md`

Dimension	Level A	Level B
Search scope	narrow	broad
Constraints	strict	permissive
Strategy hint	grid-like	exploratory
Eval criterion	single	multi-metric
Verbosity	terse	detailed

- Full factorial: $2^5 = 32$
- Fractional factorial: 8–16 runs

Study 1: Analysis Plan

Quantitative metrics

- Total experiments
- Number of improvements
- Improvement rate
- Best metric reached
- Steps to first improvement
- Diversity of hypotheses

Statistical analysis

- ANOVA
- Effect sizes

Qualitative analysis

- Parse git logs
- Code agent hypotheses
- Identify search patterns

Expected outcome

A ranked list of `program.md` dimensions by impact on autoresearch efficiency.

Study 2: Gradient Boosting

What stays fixed

- Same dataset
- Same preprocessing
- Same factorial design
- Same analysis pipeline

What changes

- GBM / XGBoost model
- Different HP space
- Stronger HP interactions

Hypotheses

1. Broad search harms GBM more than RF
2. Strategy hints matter more for GBM
3. Multi-metric evaluation improves GBM

Study 3: HP Tuning Comparison

Design

- Best `program.md` from Studies 1–2
- RF vs GBM under identical conditions
- Compare against:
  - Random Search
  - Bayesian Optimization
  - Grid Search

Central metric

Anytime performance curves

	Bayesian Opt	Autoresearch
Search space	Predefined	Open-ended
Acquisition	Surrogate	LLM reasoning
Memory	Explicit	In-context
Scope	HP only	HP + code

Research question

Does autoresearch discover novel solutions?

Systematic `program.md` Analysis

Step 1 — Parse files

- Build section trees
- Extract YAML/config blocks
- Create feature vectors

Step 2 — Experiment logs

- Parse `results.tsv`
- Extract git history
- Compute diversity metrics

Step 3 — Link program to outcomes

- Regression analysis
- SHAP values
- Similarity metrics

Step 4 — Qualitative coding

- Open coding
- Strategy archetypes
- Map triggers to behaviours

Project Timeline

Milestone	Week	Deliverable
M1	3	Environment ready + baseline run
M2	8	RF study completed
M3	11	GBM study completed
M4	13	Comparison report
M5	14	Final report submitted

Timeline overview

- Weeks 1–3: setup + literature review
- Weeks 4–8: Random Forest experiments
- Weeks 8–11: Gradient Boosting experiments
- Weeks 11–13: HP comparison
- Week 14: final report

Summary of the Research Plan

Four-study structure

1. Random Forest analysis
2. Gradient Boosting replication
3. HP tuning comparison
4. Open-ended pilot study

Known limitations

- Single dataset
- API cost limits
- Agent variability
- Open-ended tasks lack oracle

Scientific contributions

- First systematic study of `program.md`
- Mixed-methods design
- Replicable protocol

Discussion Points for Professor

1. Dataset choice
2. Scope of Study 4
3. Baselines for Study 3
4. Replication policy
5. Final write-up format

Thank you

Questions & feedback welcome